

Appendix A: Machine learning model embedding

A.1. Linear models

Linear Regression. Linear regression (LR) is a natural choice of predictive function given its inherent linearity and ease of embedding. A regression model can be trained to predict the outcome of interest, y , as a function of $\mathbf{x}$ and $\mathbf{w}$. The algorithm can optionally use regularization; the embedding only requires the final coefficient vectors $\beta_{\mathbf{x}} \in \mathbb{R}^n$ and $\beta_{\mathbf{w}} \in \mathbb{R}^p$ (and intercept term β_0) to describe the model. The model can then be embedded as

$$y = \beta_0 + \beta_{\mathbf{x}}^\top \mathbf{x} + \beta_{\mathbf{w}}^\top \mathbf{w}.$$

Support Vector Machines. A support vector machine (SVM) uses a hyper-plane split to generate predictions, both for classification (Cortes and Vapnik 1995) and regression (Drucker et al. 1997). We consider the case of *linear* SVMs, since this allows us to obtain the prediction as a linear function of the decision variables $\mathbf{x}$. In linear support vector regression (SVR), which we use for function learning, we fit a linear function to the data. The setting is similar to linear regression, but the loss function only penalizes residuals greater than an ϵ threshold (Drucker et al. 1997). As with linear regression, the trained model returns a linear function with coefficients $\beta_{\mathbf{x}}$, $\beta_{\mathbf{w}}$, and β_0 . The final prediction is

$$y = \beta_0 + \beta_{\mathbf{x}}^\top \mathbf{x} + \beta_{\mathbf{w}}^\top \mathbf{w}.$$

For the classification setting, linear support vector classification (SVC) identifies a hyper-plane that best separates positive and negative samples (Cortes and Vapnik 1995). A trained SVC model similarly returns coefficients $\beta_{\mathbf{x}}$, $\beta_{\mathbf{w}}$, and β_0 , where a sample's prediction is given by

$$y = \begin{cases} 1, & \text{if } \beta_0 + \beta_{\mathbf{x}}^\top \mathbf{x} + \beta_{\mathbf{w}}^\top \mathbf{w} \geq 0; \\ 0, & \text{otherwise.} \end{cases}$$

In SVC, the output variable y is binary rather than a probability. In this case, the constraint can simply be embedded as $\beta_0 + \beta_{\mathbf{x}}^\top \mathbf{x} + \beta_{\mathbf{w}}^\top \mathbf{w} \geq 0$.

A.2. Decision trees

Consider the leaves in Figure 2. An observation will be assigned to the leftmost leaf (node 3) if $A_1^\top \mathbf{x} \leq b_1$ and $A_2^\top \mathbf{x} \leq b_2$. An observation would be assigned to node 4 if $A_1^\top \mathbf{x} \leq b_1$ and $A_2^\top \mathbf{x} > b_2$, or equivalently, $-A_2^\top \mathbf{x} < -b_2$. Furthermore, we can remove the strict inequalities using a sufficiently small ϵ parameter, so that $-A_2^\top \mathbf{x} \leq -b_2 - \epsilon$. We can then encode the leaf assignment of observation $\mathbf{x}$ through the following constraints:

$$A_1^\top \mathbf{x} - M(1 - l_3) \leq b_1, \tag{1a}$$

$$A_2^\top \mathbf{x} - M(1 - l_3) \leq b_2, \quad (1b)$$

$$A_1^\top \mathbf{x} - M(1 - l_4) \leq b_1, \quad (1c)$$

$$-A_2^\top \mathbf{x} - M(1 - l_4) \leq -b_2 - \epsilon, \quad (1d)$$

$$-A_1^\top \mathbf{x} - M(1 - l_6) \leq -b_1 - \epsilon, \quad (1e)$$

$$A_5^\top \mathbf{x} - M(1 - l_6) \leq b_5, \quad (1f)$$

$$-A_1^\top \mathbf{x} - M(1 - l_7) \leq -b_1 - \epsilon, \quad (1g)$$

$$-A_5^\top \mathbf{x} - M(1 - l_7) \leq -b_5 - \epsilon, \quad (1h)$$

$$l_3 + l_4 + l_6 + l_7 = 1, \quad (1i)$$

$$y - (p_3 l_3 + p_4 l_4 + p_6 l_6 + p_7 l_7) = 0, \quad (1j)$$

where l_3, l_4, l_6, l_7 are binary variables associated with the corresponding leaves. For a given $\mathbf{x}$, if $A_1^\top \mathbf{x} \leq b_1$, Constraints (1e) and (1h) will force l_6 and l_7 to zero, respectively. If $A_2^\top \mathbf{x} \leq b_2$, constraint (1d) will force l_4 to 0. The assignment constraint (1i) will then force $l_3 = 1$, assigning the observation to leaf 3 as desired. Finally, constraint (1j) sets y to the prediction of the assigned leaf (p_3). We can then constrain the value of y using our desired upper bound of τ (or lower bound, without loss of generality).

More generally, consider a decision tree $\hat{h}(\mathbf{x}, \mathbf{w})$ with a set of leaf nodes $\mathcal{L}$ each described by a binary variable l_i and a prediction score p_i . Splits take the form $(A_x)^\top \mathbf{x} + (A_w)^\top \mathbf{w} \leq b$, where A_x gives the coefficients for the optimization variables $\mathbf{x}$ and A_w gives the coefficients for the non-optimization (fixed) variables $\mathbf{w}$. Let $\mathcal{S}^l$ be the set of nodes that define the splits that observations in leaf i must obey. Without loss of generality, we can write these all as $(\bar{A}_x)_j^\top \mathbf{x} + (\bar{A}_w)_j^\top \mathbf{w} - M(1 - l_i) \leq \bar{b}_j$, where $\bar{A}$ is A if leaf i follows the left split of j and $-A$ otherwise. Similarly, $\bar{b}$ equals b if the leaf falls to the left split, and $-b - \epsilon$ otherwise, as established above. This decision tree can then be embedded through the following constraints:

$$(\bar{A}_x)_j^\top \mathbf{x} + (\bar{A}_w)_j^\top \mathbf{w} - M(1 - l_i) \leq \bar{b}_j, \quad i \in \mathcal{L}, j \in \mathcal{S}^l, \quad (2a)$$

$$\sum_{i \in \mathcal{L}} l_i = 1, \quad (2b)$$

$$y - \sum_{i \in \mathcal{L}} p_i l_i = 0. \quad (2c)$$

Here, M can be selected for each split by considering the maximum difference between $(\bar{A}_x)_j^\top \mathbf{x} + (\bar{A}_w)_j^\top \mathbf{w}$ and b_j . A prescription solution $\mathbf{x}$ for a patient with features $\mathbf{w}$ must obey the constraints determined by its split path, *i.e.* only the splits that lead to its assigned leaf i . If $l_i = 0$ for some leaf i , the corresponding split constraints need not be considered. If $l_i = 1$, constraint (2a) will enforce that the solution obeys all split constraints leading to leaf i . If $l_i = 0$, no constraints

related to leaf i should be applied. When $l_i = 0$, constraint (2a) will be nonbinding at node j if $M \geq (\bar{A}_x)_j^\top \mathbf{x} + (\bar{A}_w)_j^\top \mathbf{w} - \bar{b}_j$. Thus we can find the minimum necessary value of M by maximizing these expressions over all possible values of $\mathbf{x}$ (for the patient's fixed $\mathbf{w}$). For a given patient with features $\mathbf{w}$ for whom we wish to optimize treatment, $\text{EM}(\mathbf{w})$ is the solution of

$$\max_{\mathbf{x}} (\bar{A}_x)_j^\top \mathbf{x} + (\bar{A}_w)_j^\top \mathbf{w} - \bar{b}_j \quad (3a)$$

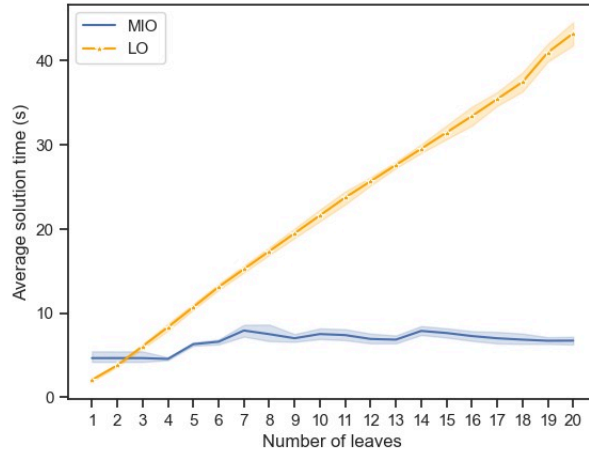
$$\text{s.t. } \mathbf{g}(\mathbf{x}, \mathbf{w}) \leq 0, \quad (3b)$$

$$\mathbf{x} \in \mathcal{X}(\mathbf{w}). \quad (3c)$$

Note that the non-learned constraints on $\mathbf{x}$, namely constraint (3b), and the trust region constraint (3c) allow us to reduce the search space when determining M .

MIO vs. LO formulation for decision trees. In Section 2, we proposed two ways of embedding a decision tree as a constraint. The first uses an LO to represent each feasible leaf node in the tree, while the second directly uses the entire MIO representation of the tree as a constraint. To compare the performance of these two approaches, we learn the palatability constraint using decision trees (CART) grown to have various numbers of leaves, and solve the optimization model with both approaches.

Figure EC.1 Comparison of MIO and multiple LO approach to tree representation, as a function of the number of leaves.



When comparing the solution times (averaged over 10 runs), Figure EC.1 shows that the MIO approach is relatively consistent in terms of solution time regardless of the number of leaves. With the LO approach however, as the number of leaves grows, the number of LOs to be solved also grows. While the solution time of a single LO is very low, solving multiple LOs sequentially might

be heavily time consuming. A way to speed up the process is to solve the LOs in parallel. When only one LO needs to be solved, it takes 1.8 seconds in this problem setting. By parallelizing the solution of the LOs, the total solution time can be expected to take only as long as it takes for the slowest LO to be solved.

A.3. Multi-layer perceptrons

MLPs consist of an input layer, $L - 2$ hidden layer(s), and an output layer. In a given hidden layer l of the network, with nodes N^l , the value of a node $i \in N^l$, denoted as v_i^l , is calculated using the weighted sum of the previous layer's node values, followed by the ReLU activation function, $\text{ReLU}(x) = \max\{0, x\}$. The value is given as

$$v_i^l = \max \left\{ 0, \beta_{i0}^l + \sum_{j \in N^{l-1}} \beta_{ij}^l v_j^{l-1} \right\},$$

where β_i^l is the coefficient vector for node i in layer l .

The ReLU operator can be encoded using linear constraints:

$$v \geq x, \tag{4a}$$

$$v \leq x - M_L(1 - z), \tag{4b}$$

$$v \leq M_U z, \tag{4c}$$

$$v \geq 0, \tag{4d}$$

$$z \in \{0, 1\}, \tag{4e}$$

where $M_L < 0$ is a lower bound on all possible values of x , and $M_U > 0$ is an upper bound. While this embedding relies on a big- M formulation, it can be improved in multiple ways. The model can be tightened by careful selection of M_L and M_U . Furthermore, [Anderson et al. \(2020\)](#) recently proposed an additional iterative cut generation procedure to improve the strength of the basic big- M formulation.

The constraints for an MLP network can be generated recursively starting from the input layer, with a set of ReLU constraints for each node in each internal layer, $l \in \{2, \dots, L - 1\}$. This allows us to embed a trained MLP with an arbitrary number of hidden layers and nodes into an MIO.

Regression. In a regression setting, the output layer L consists of a single node that is a linear combination of the node values in layer $L - 1$, so it can be encoded directly as

$$y = v^L = \beta_0^L + \sum_{j \in N^{L-1}} \beta_j^L v_j^{L-1}.$$

Binary Classification. In the binary classification setting, the output layer requires one neuron with a sigmoid activation function, $S(x) = \frac{1}{1+e^{-x}}$. The value is given as

$$v^L = \frac{1}{1 + e^{-(\beta_0^L + \boldsymbol{\beta}^{L\top} \mathbf{v}^{L-1})}}$$

with $v^L \in (0, 1)$. This function is nonlinear, and thus, cannot be directly embedded into our formulation. However, if τ is our desired probability lower bound, it will be satisfied when $\beta_0^L + \boldsymbol{\beta}^{L\top} \mathbf{v}^{L-1} \geq \ln\left(\frac{\tau}{1-\tau}\right)$. Therefore, the neural network's output, binarized with a threshold of τ , is given by

$$y = \begin{cases} 1, & \text{if } \beta_0^L + \boldsymbol{\beta}^{L\top} \mathbf{v}^{L-1} \geq \ln\left(\frac{\tau}{1-\tau}\right); \\ 0, & \text{otherwise.} \end{cases}$$

For example, at a threshold of $\tau = 0.5$, the predicted value is 1 when $\beta_0^L + \boldsymbol{\beta}^{L\top} \mathbf{v}^{L-1} \geq 0$. Here, τ can be chosen according to the minimum necessary probability to predict 1. As for the SVC case, y is binary and the constraint can be embedded as $y \geq 1$. We refer to Appendix [A.3](#) for the case of neural networks trained for multi-class classification.

Multi-class classification. In multi-class classification, the outputs are traditionally obtained by applying a *softmax* activation function, $S(\mathbf{x})_i = e^{x_i} / \left(\sum_{k=1}^K e^{x_k}\right)$, to the final layer. This function ensures that the outputs sum to one and can thus be interpreted as probabilities. In particular, suppose we have a K -class classification problem. Each node in the final layer has an associated weight vector $\boldsymbol{\beta}_i$, which maps the nodes of layer $L-1$ to the output layer by $\boldsymbol{\beta}_i^\top \mathbf{v}^{L-1}$. The softmax function rescales these values, so that class i will be assigned probability

$$v_i^L = \frac{e^{\boldsymbol{\beta}_i^\top \mathbf{v}^{L-1}}}{\sum_{k=1}^K e^{\boldsymbol{\beta}_k^\top \mathbf{v}^{L-1}}}.$$

We cannot apply the softmax function directly in an MIO framework with linear constraints. Instead, we use an *argmax* function to directly return an indicator of the highest probability class, similar to the approach with SVC and binary classification MLP. In other words, the output $\mathbf{y}$ is the identity vector with $y_i = 1$ for the most likely class. Class i has the highest probability if and only if

$$\beta_{i0}^L + \boldsymbol{\beta}_i^{L\top} \mathbf{v}^{L-1} \geq \beta_{k0}^L + \boldsymbol{\beta}_k^{L\top} \mathbf{v}^{L-1}, \quad k = 1, \dots, K.$$

We can constrain this with a big- M constraint as follows:

$$\beta_{i0}^L + \boldsymbol{\beta}_i^{L\top} \mathbf{v}^{L-1} \geq \beta_{k0}^L + \boldsymbol{\beta}_k^{L\top} \mathbf{v}^{L-1} - M(1 - y_i), \quad k = 1, \dots, K, \quad (5a)$$

$$\sum_{k=1}^K y_k = 1. \quad (5b)$$

Constraint [\(5a\)](#) forces $y_i = 0$, if the constraint is not satisfied for some $k \in \{1, \dots, K\}$. Constraint [\(5b\)](#) ensures that $y_i = 1$ for the highest likelihood class. We can then constrain the prediction to fall in our desired class i by enforcing $y_i = 1$.

A.4. Model-wrapper approach

As discussed in Section 3.1, we can embed a set of models, rather than a single model, to improve the robustness of constraint satisfaction. This ensemble of P estimators can be obtained through multiple approaches, such as through bootstrapped estimators within a single model class (e.g., P linear models or P decision trees) or by combining estimators across a range of model types (e.g., one linear model, one decision tree, and so on). Given the set of P estimators, we then constrain that at most α proportion of the estimators violate the desired constraint. The α parameter then allows us to control the degree of conservativeness of our solution, with higher α values resulting in more permissive solutions and lower α resulting in more stringent constraint requirements. In order to constrain the violation proportion, we need indicator variables to indicate whether each of the P estimators satisfies the constraint. We use a big-M formulation to obtain these indicators $z_i \forall i = 1, \dots, P$, as outlined in Section 3.1. However, this does increase the complexity of the master problem through the introduction of additional binary variables. There are two special cases of the violation limit that circumvent the need for a big-M formulation:

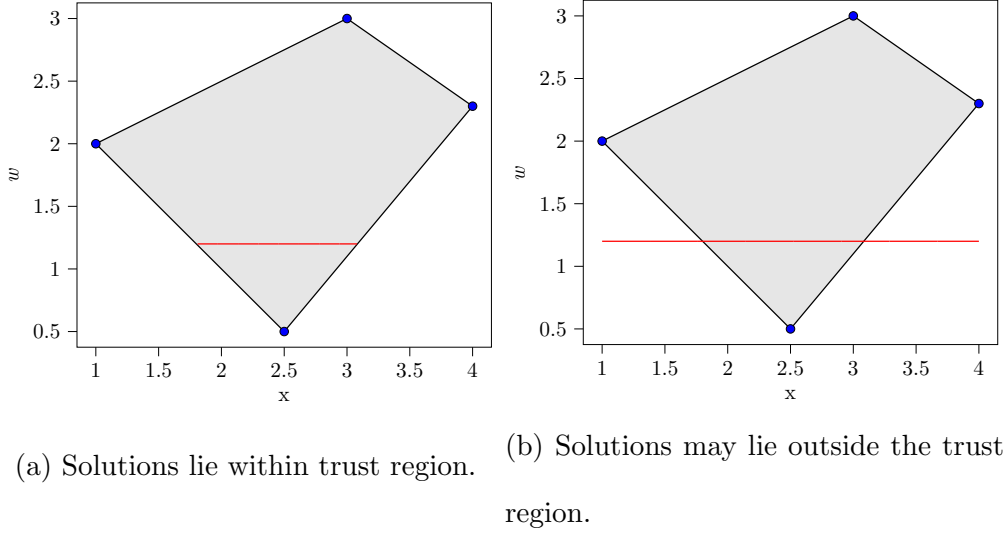
- **No allowable violation:** We can enforce a violation limit of $\alpha = 0\%$, effectively the most conservative “worst case violation” approach.
- **Average constraint:** Rather than constraining a certain proportion of estimators to obey the constraint, we can enforce that the *average* prediction of all estimators obeys the constraint. This avoids the need for tracking individual constraint satisfaction for each estimator.

In general, we note that the embedded models can be highly nonconvex on their own (e.g., if using an ensemble model as the base estimator, such as Random Forests). Thus, the additional constraints to identify and constrain violating models in this model wrapper approach are not the primary complexity drivers in the master problem, rather complexity is driven by the individual estimators. The experiments in Appendix C.3 further investigate this latter issue: we explore runtime as the number of estimators (P) increases, the incremental benefit of increasing the number of estimators, and the impact of early stopping conditions.

Appendix B: Trust region

As we explain in Section 2.3, the trust region prevents the predictive models from extrapolating. It is defined as the convex hull of the set $\mathcal{Z} = \{(\bar{\mathbf{x}}_i, \bar{\mathbf{w}}_i)\}_{i=1}^N$, with $\bar{\mathbf{x}}_i \in \mathbb{R}^n$ observed treatment decisions, and $\bar{\mathbf{w}}_i \in \mathbb{R}^p$ contextual information. In Section B.1, we explain the importance of using both $\bar{\mathbf{x}}$ and $\bar{\mathbf{w}}$ in the formulation of the convex hull. When the number of samples (N) is too large, the optimization model trust region constraints may become computationally expensive. In this case, we propose a column selection algorithm which is detailed in Section B.2.

Figure EC.2 Effect of $\bar{w}$ on the trust region.



B.1. Defining the convex hull

We characterize the feasible decision space using the convex hull of our observed data. In general, we recommend defining the feasible region with respect to both $\bar{x}$ and $\bar{w}$. This ensures that our prescriptions are reasonable with respect to the contextual variables as well. Note that for different values of w , the convex hull in the x space may be different. In Figure EC.2, the shaded region represents the convex hull of $\mathcal{Z}$ formed by the dataset (blue dots), and the red line represents the set of trusted solutions when w is fixed to a certain value. In Figure EC.2a, we see that the set of trusted solutions (red line) lies within $\text{CH}(\mathcal{Z})$ when we include $\bar{w}$. If we leave out $\bar{w}$ in the definition of the trust region, then we end up with the undesired situation shown in Figure EC.2b, where the solution may lie outside of $\text{CH}(\mathcal{Z})$. We observe that in some cases we must define the convex hull with a subset of variables. This is true in cases where the convex hull constraint leads to excessive data thinning, in which case it may be necessary to define the convex hull on treatment variables only.

B.2. Column selection

In this section, we propose a column selection method to deal with a huge set of data points. When objectives and constraints are linear, our method reduces to the Dantzig-Wolfe decomposition method (Dantzig and Wolfe 1960). However, in our case, we do not have to solve the dual problem, since we can just enumerate all the data points. In case the functions f and g in formulation (1) are nonlinear and convex the Dantzig-Wolfe method cannot be used. The key point in our approach is the choice of the dual problem. Although the use of Fenchel duality seems a logical way to deal with our problem, it appears that Wolfe duality, which in general leads to nonconvex formulations,

is exactly what we need. In (Stoer and Botkin 2005) and (Stoer et al. 2007) another method is described to optimize over the convex hull of a huge set of points. However, the method proposed in these papers is only suitable for problems that have only the convex hull constraint and no additional constraints.

Let $P_{\mathcal{I}}$ be a convex and continuously differentiable model consisting of an objective function and constraints that may be known a priori as well as learned from data. Like in Section 2.3, we denote the index set of samples by $\mathcal{I}$. As part of the constraints, the trust region is defined on the entire set $\mathcal{Z}$. We start with the matrix $\mathbf{Z} \in \mathbb{R}^{N \times (n+p)}$, where each row corresponds to a given data point in $\mathcal{Z}$. Then, model $P_{\mathcal{I}}$ is given as

$$\min_{\boldsymbol{\lambda}} f(\mathbf{Z}^\top \boldsymbol{\lambda}) \tag{6a}$$

$$\text{s.t. } g_j(\mathbf{Z}^\top \boldsymbol{\lambda}) \leq 0, \quad j = 1, \dots, m, \quad \perp \boldsymbol{\mu}, \tag{6b}$$

$$\sum_{i \in \mathcal{I}} \lambda_i = 1, \quad \perp \rho, \tag{6c}$$

$$\lambda_i \geq 0, \quad i \in \mathcal{I}, \quad \perp \mathbf{v}, \tag{6d}$$

where the decision variable $\mathbf{x}$ is replaced by $\mathbf{Z}^\top \boldsymbol{\lambda}$. Constraints (6b) include both *known* and *learned* constraints, while constraints (6c) and (6d) are used for the trust region. The dual variables associated with constraints (6b), (6c), and (6d) are $\boldsymbol{\mu} \in \mathbb{R}^m$, $\rho \in \mathbb{R}$, and $\mathbf{v} \in \mathbb{R}^N$, respectively. Note that for readability, we omit the contextual variables ($\mathbf{w}$) without loss of generality.

When we deal with huge datasets, solving $P_{\mathcal{I}}$ may be computationally expensive. Therefore, we propose an iterative column selection algorithm (Algorithm 1) that can be used to speed up the optimization while still obtaining a global optima.

The algorithm starts by initializing $\mathcal{I}' \subseteq \mathcal{I}$ with an arbitrarily small subset of samples $\mathcal{I}^0$ and iteratively solves the restricted master problem $P_{\mathcal{I}'}$ and the WOLFEDUAL function. By solving $P_{\mathcal{I}'}$, we get the primal and dual optimal solutions $\boldsymbol{\lambda}^*$ and $(\boldsymbol{\mu}^*, \rho^*, \mathbf{v}^*)$, respectively. The primal and dual optimal solutions, together with $\mathcal{I}$ and $\mathcal{I}'$, are given as input to WOLFEDUAL which returns a set of samples $\bar{\mathcal{I}} \subseteq \mathcal{I} \setminus \mathcal{I}'$ with negative reduced cost. If $\bar{\mathcal{I}}$ is not empty it is added to $\mathcal{I}'$ and a new iteration starts, otherwise the algorithm stops, and $\boldsymbol{\lambda}^*$ (with the corresponding $\mathbf{x}^*$) is returned as the global optima of $P_{\mathcal{I}}$. A visual interpretation of Algorithm 1 is shown in Figure 4.

In function WOLFEDUAL, samples $\bar{\mathcal{I}}$ are selected using the Karush–Kuhn–Tucker (KKT) stationary condition which corresponds to the equality constraint in the Wolfe dual formulation of $P_{\mathcal{I}}$ (Wolfe 1961). The KKT stationary condition of $P_{\mathcal{I}'}$ is

$$\nabla_{\boldsymbol{\lambda}} f(\tilde{\mathbf{Z}}^\top \boldsymbol{\lambda}^*) + \sum_{i=1}^m \mu_i^* \nabla_{\boldsymbol{\lambda}} g_i(\tilde{\mathbf{Z}}^\top \boldsymbol{\lambda}^*) - e\rho^* - \mathbf{v}^* = \mathbf{0}, \tag{7}$$

Algorithm 1 Column Selection

Input: $\mathcal{I}$ ▷ Index set of columns of $\mathbf{Z}^\top$

Output: λ^* ▷ Optimal solution

1: $\mathcal{I}' \leftarrow \mathcal{I}^0$ ▷ Initial column pool

2: **while** TRUE **do**

3: $\lambda^*, (\mu^*, \rho^*, \mathbf{v}^*) \leftarrow \text{P}_{\mathcal{I}'}$

4: $\bar{\mathcal{I}} \leftarrow \text{WOLFEDUAL}(\lambda^*, (\mu^*, \rho^*, \mathbf{v}^*), \mathcal{I}', \mathcal{I})$ ▷ Column(s) selection

5: **if** $\bar{\mathcal{I}} \neq \emptyset$ **then**

6: $\mathcal{I}' \leftarrow \mathcal{I}' \cup \bar{\mathcal{I}}$

7: **else**

8: Break

9: **end if**

10: **end while**

where $\tilde{\mathbf{Z}}$ is the matrix constructed with samples in $\mathcal{I}'$, and $\mathbf{e}$ is an N' -dimensional vector of ones with $N' = |\mathcal{I}'|$. Equation (7) can be rewritten as

$$\tilde{\mathbf{Z}} \nabla_{\mathbf{x}} f(\tilde{\mathbf{Z}}^\top \lambda^*) + \sum_{i=1}^m \mu_i^* \tilde{\mathbf{Z}} \nabla_{\mathbf{x}} g_i(\tilde{\mathbf{Z}}^\top \lambda^*) - \mathbf{e} \rho^* - \mathbf{v}^* = \mathbf{0}. \quad (8)$$

Equation (8) is used to evaluate the reduced cost related to each sample $\bar{\mathbf{z}} \in \mathcal{Z}$ which is not in matrix $\tilde{\mathbf{Z}}$. Consider a new sample $\bar{\mathbf{z}}$ in (8), with its associated $\lambda_{N'+1}$ set equal to zero. $(\lambda_1^*, \dots, \lambda_{N'}^*, \lambda_{N'+1}^*)$ is still a feasible solution of the restricted master problem $\text{P}_{\mathcal{I}'}$, since it does not affect the value of $\mathbf{x}$. As a consequence, μ and ρ will not change their value, nor will f and $\mathbf{g}$. The only unknown variable is $v_{N'+1}$, namely the reduced cost of $\bar{\mathbf{z}}$. However, we can write it as

$$\begin{pmatrix} \mathbf{v}^* \\ v_{N'+1} \end{pmatrix} = \begin{pmatrix} \tilde{\mathbf{Z}} \\ \bar{\mathbf{z}}^\top \end{pmatrix} \nabla_{\mathbf{x}} f(\tilde{\mathbf{Z}}^\top \lambda^*) + \sum_{i=1}^t \mu_i^* \begin{pmatrix} \tilde{\mathbf{Z}} \\ \bar{\mathbf{z}}^\top \end{pmatrix} \nabla_{\mathbf{x}} g_i(\tilde{\mathbf{Z}}^\top \lambda^*) - \mathbf{e} \rho^*. \quad (9)$$

If $v_{N'+1}$ is negative it means that we may improve the incumbent solution of $\text{P}_{\mathcal{I}'}$ by including the sample $\bar{\mathbf{z}}$ in $\tilde{\mathbf{Z}}$.

LEMMA 1. *After solving the convex and continuously differentiable problem $\text{P}_{\mathcal{I}'}$, the sample in $\mathcal{I} \setminus \mathcal{I}'$ with the most negative reduced cost is a vertex of the convex hull $\text{CH}(\mathcal{Z})$.*

Proof From equation (9) we have

$$v_{N'+1} = \bar{\mathbf{z}}^\top \nabla_{\mathbf{x}} f(\tilde{\mathbf{Z}}^\top \boldsymbol{\lambda}^*) + \bar{\mathbf{z}}^\top \nabla_{\mathbf{x}} \mathbf{g}(\tilde{\mathbf{Z}}^\top \boldsymbol{\lambda}^*) \boldsymbol{\mu}^* - \rho^*. \quad (10)$$

The problem of finding $\bar{\mathbf{z}}$, such that its reduced cost is the most negative one, can be written as a linear program where equation (10) is being minimized, and a solution must lie within $\text{CH}(\mathcal{Z})$. That is,

$$\begin{aligned} \min_{\mathbf{z}, \boldsymbol{\lambda}} \quad & \mathbf{z}^\top \nabla_{\mathbf{x}} f(\tilde{\mathbf{Z}}^\top \boldsymbol{\lambda}^*) + \mathbf{z}^\top \nabla_{\mathbf{x}} \mathbf{g}(\tilde{\mathbf{Z}}^\top \boldsymbol{\lambda}^*) \boldsymbol{\mu}^* - \rho^* \\ \text{s.t.} \quad & \mathbf{Z}^\top \boldsymbol{\lambda} = \mathbf{z}, \\ & \sum_{j \in \mathcal{I}} \lambda_j = 1, \\ & \lambda_j \geq 0, \quad j \in \mathcal{I}, \end{aligned} \quad (11)$$

where $\mathbf{z}$ and $\boldsymbol{\lambda}$ are the decision variables, and $\boldsymbol{\mu}^*$, $\boldsymbol{\lambda}^*$, ρ^* are fixed parameters. Since the objective function is linear with respect to $\mathbf{z}$, the optimal solution of (11) will necessarily be a vertex of $\text{CH}(\mathcal{Z})$. $\square$

To illustrate the benefits of column selection, consider the following convex optimization problem that we shall refer to as P_{exp} :

$$\min_{\mathbf{x}} \quad \mathbf{c}^\top \mathbf{x} \quad (12a)$$

$$\text{s.t.} \quad \log\left(\sum_{i=1}^n e^{x_i}\right) \leq t, \quad (12b)$$

$$\mathbf{A}\mathbf{x} \leq \mathbf{b}, \quad (12c)$$

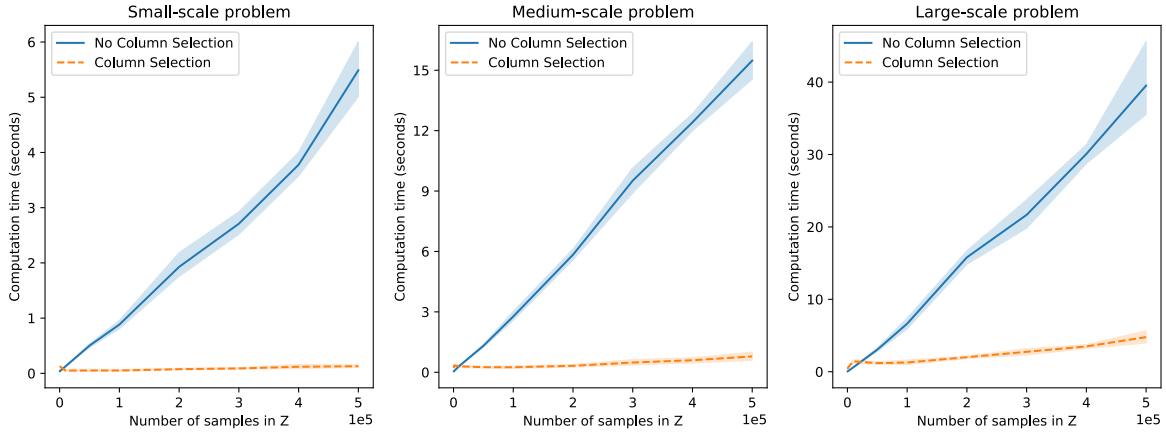
$$\sum_{i=1}^N \lambda_i \bar{\mathbf{z}}_i = \mathbf{x}, \quad (12d)$$

$$\sum_{j=1}^N \lambda_j = 1, \quad (12e)$$

$$\lambda_j \geq 0, \quad j = 1 \dots N. \quad (12f)$$

Without a loss of generality, we assume that the constraint (12b) is known a priori, and constraints (12c) are the linear embeddings of learned constraints with $\mathbf{A} \in \mathbb{R}^{k \times n}$ and $\mathbf{b} \in \mathbb{R}^k$. Constraints (12d)-(12f) define the trust region based on N datapoints. Figure EC.3 shows the computation time required to solve P_{exp} with different values of n , k , and N . The “No Column Selection” approach consists of solving P_{exp} using the entire dataset. The “Column Selection” approach makes use of Algorithm 1 to solve the problem, starting with $|\mathcal{I}^0| = 100$, and selecting only one sample at each iteration, *i.e.*, the one with the most negative reduced cost. It can be seen that in all cases, the use of column selection results in significantly improved computation times. This allows us to more quickly define the trust region for problems with large amounts of data.

Figure EC.3 Effect of column selection on computation time. Solution times are reported for three different sizes of problem P_{exp} . Small-scale: $n = 5$, $k = 10$. Medium-scale: $n = 10$, $k = 50$. Large-scale: $n = 20$, $k = 100$. The number of samples goes from 500 to 5×10^5 . In each iteration, the sample with most negative reduced cost is selected. The same problem is solved using **MOSEK** (2019) with conic reformulation for 10 different instances where c , A , and b are randomly generated.



Appendix C: WFP case study

Table EC.1 and Table EC.2 show the nutritional value of each food and our assumed nutrient requirements, respectively. The values adopted are based on the World Health Organization (WHO) guidelines (UNHCR et al. 2002).

Table EC.1 Nutritional contents per gram for different foods.

Food	Eng(kcal)	Prot(g)	Fat(g)	Cal(mg)	Iron(mg)	VitA(ug)	ThB1(mg)	RibB2(mg)	NicB3(mg)	Fol(ug)	VitC(mg)	Iod(ug)
Beans	335	20	1.2	143	8.2	0	0.5	0.22	2.1	180	0	0
Bulgur	350	11	1.5	23	7.8	0	0.3	0.1	5.5	38	0	0
Cheese	355	22.5	28	630	0.2	120	0.03	0.45	0.2	0	0	0
Fish	305	22	24	330	2.7	0	0.4	0.3	6.5	16	0	0
Meat	220	21	15	14	4.1	0	0.2	0.23	3.2	2	0	0
Corn-soya blend	380	18	6	513	18.5	500	0.65	0.5	6.8	0	40	0
Dates	245	2	0.5	32	1.2	0	0.09	0.1	2.2	13	0	0
Dried skim milk	360	36	1	1257	1	1,500	0.42	1.55	1	50	0	0
Milk	360	36	1	912	0.5	280	0.28	1.21	0.6	37	0	0
Salt	0	0	0	0	0	0	0	0	0	0	0	1000000
Lentils	340	20	0.6	51	9	0	0.5	0.25	2.6	0	0	0
Maize	350	10	4	13	4.9	0	0.32	0.12	1.7	0	0	0
Maize meal	360	9	3.5	10	2.5	0	0.3	0.1	1.8	0	0	0
Chickpeas	335	22	1.4	130	5.2	0	0.6	0.19	3	100	0	0
Rice	360	7	0.5	7	1.2	0	0.2	0.08	2.6	11	0	0
Sorghum/millet	335	11	3	26	4.5	0	0.34	0.15	3.3	0	0	0
Soya-fortified bulgur wheat	350	17	1.5	54	4.7	0	0.25	0.13	4.2	74	0	0
Soya-fortified maize meal	390	13	1.5	178	4.8	228	0.7	0.3	3.1	0	0	0
Soya-fortified sorghum grits	360	360	1	40	2	0	0.2	0.1	1.7	50	0	0
Soya-fortified wheat flour	360	16	1.3	211	4.8	265	0.66	0.36	4.6	0	0	0
Sugar	400	0	0	0	0	0	0	0	0	0	0	0
Oil	885	0	100	0	0	0	0	0	0	0	0	0
Wheat	330	12.3	1.5	36	4	0	0.3	0.07	5	51	0	0
Wheat flour	350	11.5	1.5	29	3.7	0	0.28	0.14	4.5	0	0	0
Wheat-soya blend	370	20	6	750	20.8	498	1.5	0.6	9.1	0	40	0

Eng = Energy, Prot = Protein, Cal = Calcium, VitA = Vitamin A, ThB1 = ThiamineB1, RibB2 = RiboflavinB2, NicB3 = NicacinB3, Fol = Folate, VitC = Vitamin C, Iod = Iodine

Table EC.2 Nutrient requirements used in optimization model.

Type	Eng(kcal)	Prot(g)	Fat(g)	Cal(mg)	Iron(mg)	VitA(ug)	ThB1(mg)	RibB2(mg)	NicB3(mg)	Fol(ug)	VitC(mg)	Iod(ug)
Avg person day	2100	52.5	89.25	1100	22	500	0.9	1.4	12	160	0	150

Eng = Energy, Prot = Protein, Cal = Calcium, VitA = Vitamin A, ThB1 = ThiamineB1, RibB2 = RiboflavinB2, NicB3 = NicacinB3, Fol = Folate, VitC = Vitamin C, Iod = Iodine

C.1. Food baskets generation and palatability function

Referring to Peters et al. (2021), a food basket $x_k (\forall k \in \mathcal{K})$ is defined as a collection of $\mathcal{K}$ commodities, such as beans, meat, and oil, along with their respective quantities measured in grams. These commodities are classified into five macro-categories: cereals and grains, pulses and vegetables, oils and fats, mixed and blended foods, and meat and fish, as well as dairy. Each macro-category $g \in \mathcal{G}$ is associated with an upper bound (max_g) and a lower bound (min_g), as shown in Table EC.3. We use the notation $\mathcal{K}_g$ to indicate the set of commodities belonging to category g . In contrast to the approach used in Peters et al. (2021), where bound constraints were utilized to ensure the palatability of the food basket, we expand the notion of palatability by incorporating a palatability score that ranges from non-negative values and tends towards zero for more enjoyable diets. The score is calculated as:

$$Palatability\ Score = \sqrt{\sum_{g \in \mathcal{G}} (\gamma_g (\hat{x}_g - Opt_g))^2}, \quad (13)$$

where

$$\hat{x}_g = \sum_{k \in \mathcal{K}_g} x_k \quad \text{with } g \in \mathcal{G} \text{ and}$$

$$Opt_g = \frac{max_g + min_g}{2} \quad \text{with } g \in \mathcal{G}.$$

To account for the different range sizes ($max_g - min_g$) across the macro-categories, we introduce a scaling parameter γ_g that determines their influence on the score, as presented in Table EC.3. The resulting score is normalized on a scale of 0 to 1, where a score of 1 represents a perfectly appetizing food basket, while a score of 0 indicates an inedible basket.

Table EC.3 Macro-categories bounds and scaling factor.

macro-category	min	max	γ
Cereals & Grains	200	600	1
Pulses & Vegetables	30	100	5.7
Oils & Fats	15	40	16
Mixed & Blended Foods	0	90	4.4
Meat & Fish & Dairy	0	60	6.6

The generation of diverse food baskets is done by solving several diet problems whose cost function changes at each run and enforcing constraints on the nutrient requirements as well as on the maximum number of foods belonging to the same category.

C.2. Predictive models

Table EC.4 shows the structure of the predictive models used in the WFP experiments. For each model, the choice of parameters is based on a cross-validation procedure.

Table EC.4 Definition of the predictive model parameters used in the WFP case study

Model	Parameters
Linear	ElasticNet parameters: 0.1 (alpha), 0.1 (ℓ_1 -ratio)
SVM	regularization parameter: 100
CART	max depth: 10, max features: 1.0, min samples leaf: 0.02
RF	max depth : 4, max features: auto, number of estimators: 25
GBM	learning rate: 0.2, max depth: 5, number of estimators: 20
MLP	hidden layers: 1, size hidden layers: (100,) activation: relu

C.3. Effect of robustness parameters

Robustness impact by algorithm. Table EC.5 reports the change in objective value (cost) and constrained outcome (palatability) between the nominal and bootstrapped solution with 10 estimators and a violation limit of 25%. The goal of the WFP case study is to minimize cost such that palatability is at least 0.5; thus, a smaller cost and larger palatability are better. As expected, the robust solution increases both the cost and palatability of the prescribed diets. We see that the relative increase in cost is consistently lower than the relative increase in real palatability across all methods, indicating that the improvement in palatability exceeds the incremental cost addition. While the acceptable trade off between cost and palatability could differ by use case, this could be further explored with alternative violation limits. Additionally, we compare the single algorithm constraints against an ensemble of all six methods, also with a violation limit of $\alpha = 0.25$. The ensemble with multiple algorithms yields an objective value of 1313 and real palatability of 0.57. This represents a -1.8% to 1% increase in cost and 5.6% to 15.6% increase in real palatability over the nominal solutions. When compared to the bootstrapped single-method models, it is generally *more* conservative. This is consistent with the fact that it must satisfy the constraint estimate across the majority of the individual methods, forcing it to be conservative relative to this set.

Table EC.5 Change in cost and palatability from nominal to bootstrapped ($P = 10, \alpha = 0.25$) solution.

Algorithm	Objective value			Real palatability		
	Nominal	Bootstrapped	Change	Nominal	Bootstrapped	Change
Linear	1337	1359	1.6%	0.496	0.512	3.1%
SVM	1306	1308	0.1%	0.541	0.548	1.2%
CART	1301	1307	0.5%	0.539	0.550	2.1%
RF	1305	1306	0.0%	0.543	0.551	1.5%
GBM	1300	1304	0.3%	0.532	0.553	3.9%
MLP	1307	1313	0.5%	0.537	0.587	9.4%

Effect of number of estimators. Table EC.6 compares the runtime as the number of estimators (P) increases up to 25 estimators. We see that the solve time for the linear, SVM, CART, and MLP models are stable as the number of estimators increases. In contrast, we see that the ensemble algorithms, RF and GBM, have exponential runtime increases as the number of estimators grows. RF and GBM are already comprised of multiple individual learners, so embedding multiple estimators involves adding multiple *sets* of decision trees, which becomes computationally expensive. All results are reported over 100 instances. The experiments were run using a virtual computing environment with 4 CPU and 32 GB total RAM. We also report the runtime for an ensemble of estimators obtained from different model classes (“Ensemble”), using a single model from each class.

We further investigate the runtimes with 25 estimators in Table EC.7. The left side of the table reports the mean, median, and maximum runtimes for each method on the same 100 experiments as above. We see that the RF and GBM models have reasonable median solve times (6.66 and 18.80 minutes, respectively), but the average solve times are driven up by outlier instances that have significantly higher runtimes (max. 2110 and 1603 minutes, respectively). We propose to use a time limit to control the experiment times. On the right side of the table, we see that using a 4 hour time limit returns optimal solutions for 95% of the RF runs and 82% of the GBM runs, and feasible solutions for all but four GBM instances. In cases where an optimal solution is obtained, the average runtime is less than 40 minutes. In cases where the time limit is hit, the average remaining MIP gap is 1.02% for RF and 5.21% for GBM. The results suggest that imposing this termination condition results in high quality solutions with a modest optimality gap.

The runtime experiments raise a natural question: what is the impact of embedding a larger number of estimators? We consider the cost-palatability trade off for a decision tree model as we vary the number of estimators from $P = 2$ to $P = 50$, averaged over the candidate violation limits. The results are shown in Figure EC.4. As the number of estimators increases, the results tend to be more conservative. By 10 estimators, the trade off curve well-approximates the curves for higher estimator up to an inflection point where average cost increases significantly. By $P = 25$

Table EC.6 Optimization solver runtime (minutes) as a function of number of bootstrapped estimators.

Algorithm	P = 2	P = 5	P = 10	P = 25
Linear	0.01	0.01	0.02	0.01
SVM	0.01	0.01	0.02	0.01
CART	0.02	0.02	0.03	0.12
RF	0.15	1.34	11.93	44.87
GBM	0.37	3.58	10.71	133.13
MLP	0.01	0.02	0.04	0.48
Ensemble	0.09			

Table EC.7 Runtime results for $P = 25$ estimators, both when solved to optimality (left) and with a 4 hour time limit (right).

Runtime (mins) to optimality				4 hour time limit			
Algorithm	Mean	Median	Max	% feasible	% optimal	Avg. runtime to optimality (mins)	Avg. remaining MIP gap
Linear	0.01	0.01	0.04	100%	100%	0.01	–
SVM	0.01	0.01	0.04	100%	100%	0.01	–
CART	0.12	0.08	0.64	100%	100%	0.12	–
RF	44.87	6.66	2109.72	100%	95%	19.76	1.02%
GBM	133.13	18.80	1603.51	96%	82%	37.79	5.21%
MLP	0.49	0.12	6.89	100%	100%	0.48	–

and $P = 50$, the curves closely match, suggesting diminishing value in increasing the number of estimators beyond a certain point.

Finally, Table EC.8 reports the parameters used in our bootstrapped models. For each method, we report the parameter grid that was used in our model training and selection procedure. Individual estimators use different combinations of these parameters based on the validation performance on the specific bootstrapped samples. We note that for these experiments, we used a default parameter grid implemented in OptiCL; this grid can be manually set by a user when specifying each outcome of interest before model training.

Appendix D: Chemotherapy regimen design

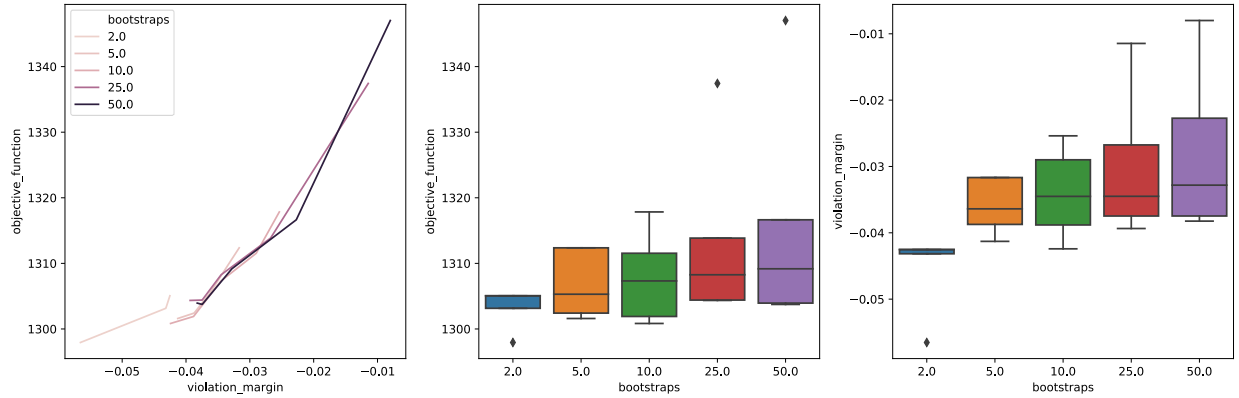
D.1. Data Processing

The data for this case study includes three components, study cohort characteristics ($\mathbf{w}$), treatment variables ($\mathbf{x}$), and outcomes ($\mathbf{y}$). The raw data was obtained from Bertsimas et al. (2016), in which the authors manually curated data from 495 clinical trial arms for advanced gastric cancer. Our feature space was processed as follows:

Cohort Characteristics. We included several cohort characteristics to adjust for the study context: fraction of male patients, median age, primary site breakdown (Stomach vs. GEJ), fraction of patients receiving prior palliative chemotherapy, and mean ECOG score. We also included variables

Table EC.8 Default parameter grid for supported algorithms.

Algorithm	Parameter Grid
Linear	alpha': [0.1, 1, 10, 100, 1000], 'l1_ratio': np.arange(0.1, 1.0, 0.2)
SVM	'C': [.1, 1, 10, 100]
CART	max_depth': [3, 4, 5, 6, 7, 8, 9, 10], 'min_samples_leaf': [0.02, 0.04, 0.06], 'max_features': [0.4, 0.6, 0.8, 1.0]
RF	n_estimators': [10, 25], 'max_features': ['auto'], 'max_depth': [2, 3, 4]
GBM	learning_rate': [0.01, 0.025, 0.05, 0.075, 0.1, 0.15, 0.2], 'max_depth': [2, 3, 4, 5], 'n_estimators': [20]
MLP	'hidden_layer_sizes': [(10,), (20,), (50,), (100,)]

Figure EC.4 Effect of the number of bootstrapped estimators (P) on the cost and palatability of the prescribed diet.

for the study context: the study year, country, and number of patients. Missing data was imputed using multiple imputation based on the other contextual variables; 20% of observations had one missing feature and 6% had multiple missing features.

Treatment Variables. Chemotherapy regimens involve multiple drugs being delivered at potentially varied frequencies over the course of a chemotherapy cycle. As a result, multiple dimensions of the dosage must be encoded to reflect the treatment strategy. As in [Bertsimas et al. \(2016\)](#), we include three variables to represent each drug: an indicator (1 if the drug is used in the regimen), instantaneous dose, and average dose.

Outcomes. We use Overall Survival (OS) as our survival metric, as reported in the clinical trials. Any observations with unreported OS are excluded. We consider several “dose-limiting toxicities” (DLTs): Grade 3/4 constitutional, gastrointestinal, infection, and neurological toxicities, as well

Table EC.9 Comparison of out-of-sample R^2 all considered models for learned outcomes in chemotherapy regimen selection problem.

Outcome	Linear	SVM	CART	RF	GBM
Any DLT	0.268	-0.094	-0.016	0.152	0.202
Blood	0.196	-1.102	0.012	0.153	0.105
Constitutional	0.106	0.144	0.157	0.194	0.136
Infection	0.082	-0.511	-0.222	0.070	0.035
Gastrointestinal	0.141	-0.196	-0.023	0.066	0.083
Overall Survival	0.448	0.385	0.474	0.496	0.450

as Grade 4 blood toxicities. The toxicities reported in the original clinical trials are aggregated according to the CTCAE toxicity classes (Cancer Therapy Evaluation Program [2006]). We also include a variable for the occurrence of any of the four individual toxicities (t_i for each toxicity $i \in T$, called DLT proportion; we treat these toxicity groups as independent and thus define the DLT proportion as

$$DLT = 1 - \prod_{i \in T} (1 - t_i).$$

We define Grade 4 blood toxicity as the maximum of five individual blood toxicities (related to neutrophils, leukocytes, lymphocytes, thrombocytes, anemia). Observations missing all of these toxicities were excluded; entries with partial missingness were imputed using multiple imputation based on other blood toxicity columns. Similarly, observations with no reported Grade 3/4 toxicities were excluded; those with partial missingness were imputed using multiple imputation based on the other toxicity columns. This exclusion criteria resulted in a final set of 461 (of 495) treatment arms.

We split the data into training/testing sets temporally. The training set consists of all clinical trials through 2008, and the testing set consists of all 2009-2012 trials. We exclude trials from the testing set if they use new drugs not seen in the training data (since we cannot evaluate these given treatments). We also identify sparse treatments (defined as being only seen once in the training set) and remove all observations that include these treatments. The final training set consists of 320 observations, and the final testing set consists of 96 observations.

D.2. Predictive Models

Table EC.9 shows the out-of-sample performance of all considered methods in the model selection pipeline. We note that model choice is based on the 5-fold validation performance, so it does not necessarily correspond to the highest test set performance. The final parameters for each model and each outcome, selected through the cross-validation procedure, are shown in Table EC.10

Table EC.10 Predictive model parameters used in the chemotherapy case study.

Outcome	Model	Parameters
Any DLT	GBM	learning rate: 0.2, max depth: 2, number of estimators: 20
Blood	Linear	ElasticNet parameters: 0.1 (alpha), 0.7 (ℓ_1 -ratio)
Constitutional	RF	max depth : 4, max features: 'auto', number of estimators: 25
Infection	Linear	ElasticNet parameters: 1 (alpha), 0.5 (ℓ_1 -ratio)
Gastrointestinal	GBM	learning rate: 0.1, max depth: 4, number of estimators: 20
Overall Survival	GBM	learning rate: 0.1, max depth: 3, number of estimators: 20

Table EC.11 Performance (R^2) of individual models in ground truth ensemble for model evaluation.

outcome	Linear	SVM	CART	RF	GBM	XGB
Any DLT	0.301	0.330	0.250	0.573	0.670	0.323
Blood	0.287	0.351	0.211	0.701	0.813	0.446
Constitutional	0.139	0.224	0.246	0.602	0.682	0.285
Infection	0.217	0.303	0.139	0.514	0.588	0.247
Gastrointestinal	0.201	0.328	0.238	0.563	0.733	0.475
Overall Survival	0.528	0.469	0.421	0.815	0.827	0.756

D.3. Prescription Evaluation

Table EC.11 shows the performance of the models that comprise the ground truth ensemble used in the evaluation framework. These models trained on the full data. We see that the ensemble models, particularly RF and GBM, have the highest performance. These models are trained on more data and include more complex parameter options (*e.g.*, deeper trees, larger forests) since they are not required to be embedded in the MIO and are rather used directly to generate predictions. The final parameters for each model and each outcome, selected through the cross-validation procedure, are shown in Table EC.12. For this reason, the GT ensemble could also be generalized to consider even broader method classes that are not directly MIO-representable, such as neural networks with alternative activation functions, providing an additional degree of robustness.

D.4. Optimization runtimes

Table EC.13 reports the runtimes of the optimization model results presented in Section 5.5, Table 6. Results are averaged over all patients in the test set.

Appendix E: Comparison with JANOS and EML

As mentioned earlier in Section 1.1, JANOS and EML are two software frameworks for embedding learned ML models in optimization problems. In this section, we compare the performance of OptiCL to those of JANOS and EML using the test problems in Bergman et al. (2022) and Lombardi et al. (2017), respectively. The experiments are conducted using an Intel i7-8665U 1.9 GHz CPU, 16 GB RAM (Windows 10 environment).

Table EC.12 Predictive model parameters used in the ground truth ensemble for model evaluation.

Algorithm	Parameter	Any DLT	Blood	Const.	Inf.	GI	OS
<i>Linear</i>	alpha	0.1	0.1	1	1	1	0.1
	ℓ_1 ratio	0.6	0.5	0.4	0.3	0.7	0.8
<i>SVM</i>	regularization parameter	100	100	1	10	100	0.1
<i>CART</i>	max depth	3	3	4	3	5	3
	max features	1	1	0.6	0.6	0.6	0.8
	min samples per leaf	0.04	0.06	0.06	0.06	0.06	0.02
<i>RF</i>	max depth	6	8	6	6	6	8
	max features	auto	auto	auto	auto	auto	auto
	number of estimators	500	500	500	250	250	250
<i>GBM</i>	learning rate	0.01	0.025	0.01	6	0.01	0.01
	max depth	5	5	5	auto	6	5
	number of estimators	250	250	250	250	250	250
<i>XGB</i>	cols sampled by tree	0.8	1	0.8	1	0.8	1
	gamma	0.5	0.5	1	1	0.5	10
	max depth	4	5	4	4	5	4
	min child weight	10	1	10	10	1	10
	number of estimators	250	250	250	250	250	250
	subsample	1	0.8	0.8	0.8	0.8	1

Const. = Constitutional, Inf. = Infection, GI = Gastrointestinal, OS = Overall survival.

Table EC.13 Average (and standard deviation) of runtimes for gastric cancer case, in seconds.

Model Version	Average Time (SD)
All Constraints	0.511 (0.892)
DLT Only	0.203 (0.433)

E.1. OptiCL vs JANOS

In the Student Enrolment Problem (SEP) in [Bergman et al. \(2022\)](#), a university’s admission office seeks to offer scholarships to some of the admitted students in order to bolster the class profile. The objective is to maximize the expected class size subject to budget constraints. This problem is formulated as:

$$\max \sum_{i=1}^N y_i \quad (14a)$$

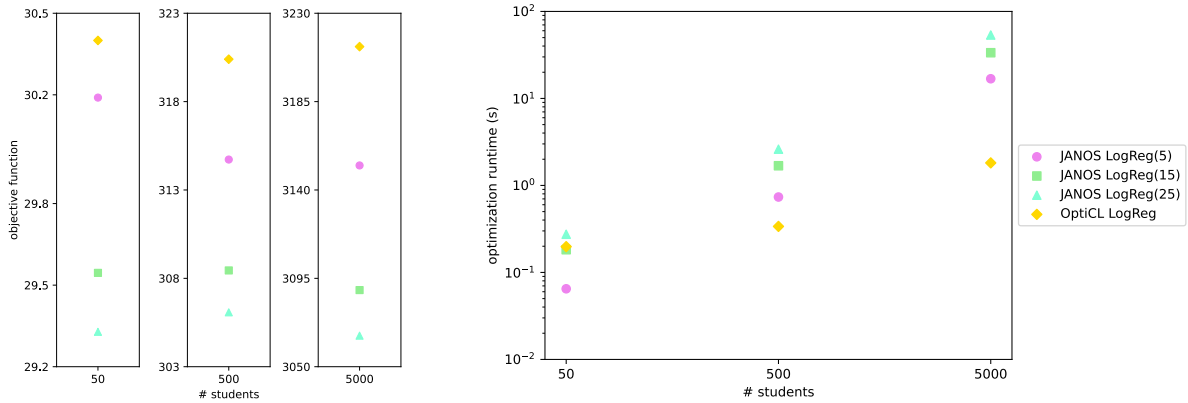
$$\text{s.t. } \sum_{i=1}^N x_i \leq \mathbf{BUDGET}, \quad (14b)$$

$$y_i = \hat{h}(s_i, g_i, x_i) \quad \forall i \in \{1, \dots, N\}, \quad (14c)$$

$$0 \leq x_i \leq 25,000 \quad \forall i \in \{1, \dots, N\}, \quad (14d)$$

where x_i is the decision variable indicating the amount of scholarship assigned to each student accepted, s_i is the SAT score of applicant i , and g_i is the GPA score of applicant i . The predicted outcome y_i represents the probability of a candidate i accepting the offer, and $\hat{h}$ is the fitted model used to predict any candidate’s probabilities of accepting an offer. The parameters s_i , g_i , and the decision variable x_i are the predictive model’s inputs. In order to compare OptiCL and JANOS, we solved the SEP for different student sizes, and compared the objective values and runtimes. Although OptiCL and JANOS handle neural network embedding in a similar manner, JANOS uses a parameterized discretization to handle logistic regression predictions. We therefore compared their performances only using the logistic regression models, as we expected to see a difference in performance based on the differences in implementation. In the experiments reported in Figure EC.5, we discretize the logistic regression (LogReg) in JANOS using three different number of intervals (reported between brackets in the Figure legend). From the experiments, we can see that OptiCL achieves better objective values in all three instances. It can also be seen that for the larger problems, OptiCL is much more efficient in terms of optimization runtime than JANOS.

Figure EC.5 Objective value (right) and runtime (left) comparison between OptiCL and JANOS for the SEP.



E.2. OptiCL vs EML

In the thermal-aware Workload Dispatching Problem (WDP) in Lombardi et al. (2017), the goal is to assign jobs to the different cores on a multi-core processor. The processor has 24 dual-core tiles arranged in a 4×6 grid, resulting in an arrangement with 48 cores in an 8×6 grid. A direct comparison between OptiCL and EML is not possible, as Lombardi et al. (2017) do not use neural networks or decision trees for constraint learning in MIO problems. Their focus for these predictive models are Local Search, Constraint Programming, or SAT Modulo Theory problems. What we do, however, is demonstrate that OptiCL is able to solve the example in an MIO setting. The model considered here is the “ANN1” model in Lombardi et al. (2017) given as:

$$\max z \quad (15a)$$

$$\text{s.t. } z \leq y_k \quad \forall k = 0, \dots, m-1, \quad (15b)$$

$$y_k = \hat{h}_k(\text{avgcpi}_k, \text{neighcpi}_k, \text{othercpi}_k) \quad \forall k = 0, \dots, m-1, \quad (15c)$$

$$\sum_{k=0}^{m-1} x_{ik} = 1 \quad \forall i = 0, \dots, n-1, \quad (15d)$$

$$\sum_{i=0}^{n-1} x_{ik} = \frac{n}{m} \quad \forall k = 0, \dots, m-1, \quad (15e)$$

$$\text{avgcpi}_k = \frac{1}{m} \sum_{i=0}^{n-1} \text{cpi}_i x_{ik} \quad \forall k = 0, \dots, m-1, \quad (15f)$$

$$\text{neighcpi}_k = \frac{1}{m} \sum_{h \in N(k)} \text{avgcpi}_h \quad \forall k = 0, \dots, m-1, \quad (15g)$$

$$\text{othercpi}_k = \frac{1}{m-1-|N(k)|} \sum_{h \neq k, h \notin N(k)} \text{avgcpi}_h \quad \forall k = 0, \dots, m-1, \quad (15h)$$

$$x_{ik} \in \{0, 1\} \quad \forall i = 0, \dots, n-1 \quad \forall k = 0, \dots, m-1, \quad (15i)$$

where x_{ik} is the binary decision variable indicating if a job i is mapped on core k or not. The parameter cpi_i represents the average Clock Per Instructions (CPI) characterizing job i , and is a measure of the difficulty of job i . The objective is to maximize the worst-case core efficiency, and the fitted model $\hat{h}_k$ is used to predict the efficiency of core k that is represented by $y_k \in [0, 1]$.

Constraints (15d) ensures that each job is mapped to only one core, and (15e) forces the same number of jobs to run on each core. Constraints (15f), (15g) and (15h) are used to compute the average CPI for a core k , the average CPI for the cores in the neighborhood of k ($N(k)$), and the average CPI for cores not in the neighborhood of k respectively. Lombardi et al. (2017) conclude that learning the efficiency function for each core by means of neural networks (with one hidden layer of two nodes and tanh activation function) is computationally intractable. On the contrary, our experiments show that we are able to solve this problem using neural networks with one hidden layer and 10 nodes in a reasonable amount of time (19.4 seconds). We tried deeper neural networks, but the increase in computational complexity did not lead to a gain in predictive performance.